

Neural Network Weight Optimization as a Constrained NLP

Arindam

Numerical Optimization

Albert-Ludwigs-Universität Freiburg

Summer Term 2026 · Prof. Dr. Moritz Diehl

July 2026

Abstract

Neural-network training is almost universally treated as unconstrained first-order optimization. This report instead casts the training of a one-hidden-layer tanh network as a constrained nonlinear program (NLP) over the flattened weights, with the mean-squared error as objective and three structural constraints: a bilinear Lipschitz bound, a convex weight norm-ball, and linear symmetry-breaking inequalities. The NLP is solved exactly with the interior-point solver IPOPT and with sequential quadratic programming through CasADi, and compared against an unconstrained Adam baseline and a penalty method. We study how the Lipschitz bound trades training fit for generalization, read the constraint's shadow price from the KKT multipliers, and measure how solve cost scales with network size. The central finding is that hard constraints act as a controllable, exactly-enforced regularizer that an unconstrained penalty only approximates.

1. Introduction

Training a neural network is, formally, an optimization problem: choose the weights that minimize a loss over the data. In practice that problem is almost always handed to a stochastic first-order method such as Adam, which is cheap, scalable, and indifferent to the geometry of the loss surface. From a numerical-optimization standpoint this is a missed opportunity. The same problem can be written as a nonlinear program (NLP) and handed to a second-order constrained solver, which lets us add *hard* structural requirements -- bounds the network must satisfy exactly, not merely on average -- and which exposes rich diagnostic information (Lagrange multipliers, Hessian conditioning, iteration counts) that gradient descent never produces.

This is interesting precisely because the resulting NLP is nontrivial: the most natural capacity constraint, a Lipschitz bound on the network, is bilinear and therefore nonconvex, so the feasible set is not convex and multiple local optima can exist. We build the problem symbolically with CasADi [1], which supplies exact first and second derivatives by automatic differentiation, and solve it with the interior-point method IPOPT [2]. The point is not to replace Adam at scale but to understand what is gained, and at what cost, by enforcing constraints exactly.

Concretely, this report addresses four research questions:

RQ1. How do an exact constrained solver (IPOPT), an SQP method, and an unconstrained baseline (Adam) compare in solution quality and cost on the identical problem?

RQ2. How does tightening a hard Lipschitz bound trade training fit against generalization, and when does the constraint actually bind?

RQ3. How does the cost of the exact solve scale with the dimension of the NLP (the number of network weights)?

RQ4. How does enforcing a constraint exactly compare with the standard machine-learning practice of adding a soft penalty term to the loss?

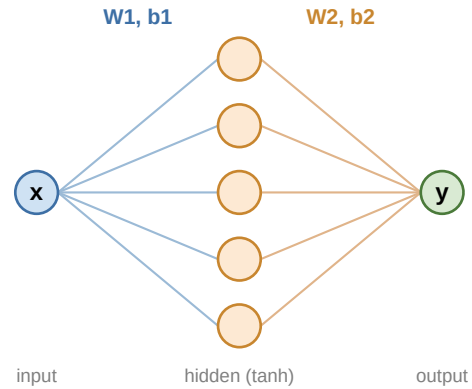
2. Problem Formulation

The student network is a single hidden layer with tanh activation, $f(x; w) = W_2 \tanh(W_1 x + b_1) + b_2$. All weights and biases are stacked into a single decision vector $w \in \mathbb{R}^n$ (the flattened W_1, b_1, W_2, b_2); for the default eight-unit network this is $n = 25$ variables. The objective is the mean-squared

training error

$$f(w) = (1/N) \sum_i (f(x_i; w) - y_i)^2,$$

minimized subject to up to three independently switchable constraints, described below. Figure 1 sketches the network whose weights are the decision variables.



Sketch 1. The one-hidden-layer student network. The weight and bias groups $W1, b1$ and $W2, b2$ form the decision vector w .

(i) Lipschitz bound. A tanh unit is 1-Lipschitz, so the network's Lipschitz constant is bounded by the product of the layer operator norms. Using the tractable Frobenius proxy gives $\|W_1\|_F^2 \cdot \|W_2\|_F^2 \leq L_{\max}^2$. This is a product of two convex terms and is therefore *bilinear and nonconvex*: the feasible set is not convex, the exact Hessian of the Lagrangian is indefinite, and the problem can admit several local minima. It is the constraint that makes this NLP genuinely hard and is the main object of study.

(ii) Weight norm-ball. A hard L2 ball on all weights, $\|w\|_2^2 \leq B_{\max}^2$. This is convex (a quadratic ball constraint) and is the exactly-enforced analogue of L2 weight-decay regularization: rather than penalizing large weights softly in the loss, it forbids them outright.

(iii) Symmetry-breaking. The hidden units of a one-layer network can be permuted without changing the function, so every minimizer has $H!$ equivalent relabelings. Ordering the hidden biases, $b_1[0] \leq b_1[1] \leq \dots \leq b_1[H-1]$, adds $H-1$ linear inequalities that remove this combinatorial degeneracy without restricting the representable function class, which helps the solver converge.

3. Methods

IPOPT (interior-point). The constrained NLP is solved primarily with IPOPT [2], a primal-dual interior-point (barrier) method. Inequality constraints are handled through a logarithmic

barrier whose weight is driven to zero across outer iterations, while each inner step solves a Newton system built from the exact Hessian of the Lagrangian supplied by CasADi. This yields fast local convergence and, as a by-product, the KKT multipliers used later for shadow-price analysis.

SQP via CasADi. As a second constrained solver we use CasADi's *sqpmethod*, a sequential quadratic programming scheme that solves a convex quadratic subproblem at each iterate. A limited-memory BFGS approximation keeps those subproblems convex, since the true Hessian is indefinite under the bilinear Lipschitz constraint. SQP reaches the same optima but, lacking the exact Hessian, typically needs far more iterations.

Adam baseline (unconstrained). As the machine-learning reference we run plain Adam on the same MSE objective with no constraints. Its gradient is taken from the identical CasADi expression, so the only differences are the algorithm and the absence of constraints. A penalty variant adds a soft hinge on the Lipschitz violation to study the penalty-versus-hard-constraint question.

Teacher-student data. All experiments use a synthetic teacher-student regression task: a fixed random one-hidden-layer network (the teacher) generates a smooth ground-truth function, sampled on $[-3, 3]$ and corrupted with Gaussian label noise. The student network must recover it from 60 noisy training points, with 40 held out for testing. This gives a known, controllable problem so that observed behaviour reflects the optimizer and the constraints, not data artefacts.

4. Results

4.1 Optimizer comparison. Table 1 reports the three core optimizers on the identical eight-unit problem ($n = 25$ variables, nine active constraint rows for the constrained solvers). IPOPT and SQP reach essentially the same fit -- train MSE 0.00185 and 0.00183 respectively -- but IPOPT needs only 277 iterations against SQP's 2146, because IPOPT uses the exact Hessian whereas SQP relies on a BFGS approximation. Adam, solving the easier unconstrained problem, reaches a comparable test MSE (0.00289) but its solution is a different point that does not respect the hard constraints. The numbers below are read directly from results/summary.csv.

Method	train MSE	test MSE	iterations	solve time [s]
IPOPT	0.00185	0.00310	277	0.451
SQP	0.00183	0.00310	2146	2.504
IPOPT (spectral)	0.00183	0.00308	513	2.605
ADAM	0.00216	0.00289	3000	0.952

Table 1. Method comparison group, read from results/summary.csv (IPOPT, SQP, exact-spectral IPOPT, Adam).

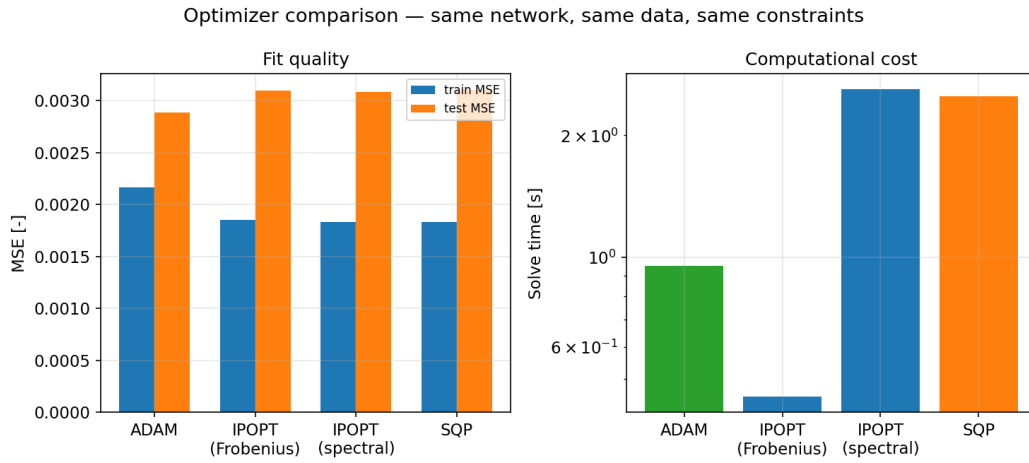


Fig. 1. IPOPT vs SQP vs Adam: train/test MSE and solve time. IPOPT and SQP solve the same constrained NLP to similar quality; Adam solves a different (unconstrained) problem.

4.2 Lipschitz bound sweep. Sweeping the bound from $L_{\max} = 32$ down to 0.5 steadily trades fit for smoothness: training MSE rises from 0.00507 at the loosest bound to 0.00976 at the tightest, as the network is denied capacity. The held-out error moves the other way -- test MSE falls from 0.01830 to 0.01115 -- so the constraint behaves as a tunable regularizer. The achieved Lipschitz value tracks the bound almost exactly at every setting (e.g. 0.500 at $L_{\max} = 0.5$), confirming the constraint is active across the whole range.

Effect of tightening the Lipschitz constraint (IPOPT)

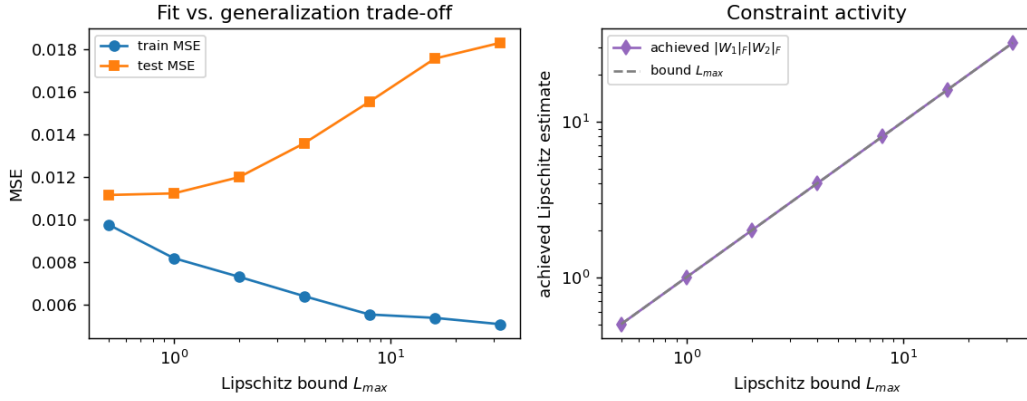


Fig. 2. Lipschitz bound sweep. As L_{max} tightens, train MSE rises (less capacity) while test MSE falls (better generalization). The constraint is active (binding) across the full range.

4.3 KKT shadow price. IPOPT returns a Lagrange multiplier (dual variable) for the Lipschitz constraint -- its shadow price, the marginal objective improvement per unit relaxation of the bound. The multiplier is strictly positive throughout, largest at the tightest bound ($\lambda \approx 2.04e-03$ at $L_{max} = 0.5$) and smallest at the loosest ($\lambda \approx 3.44e-08$ at $L_{max} = 32$). A positive multiplier with complementary slackness is the formal certificate that the constraint is binding: tightening it genuinely costs fit, exactly the trade-off seen in Section 4.2.

KKT dual-variable analysis of the Lipschitz constraint (IPOPT)

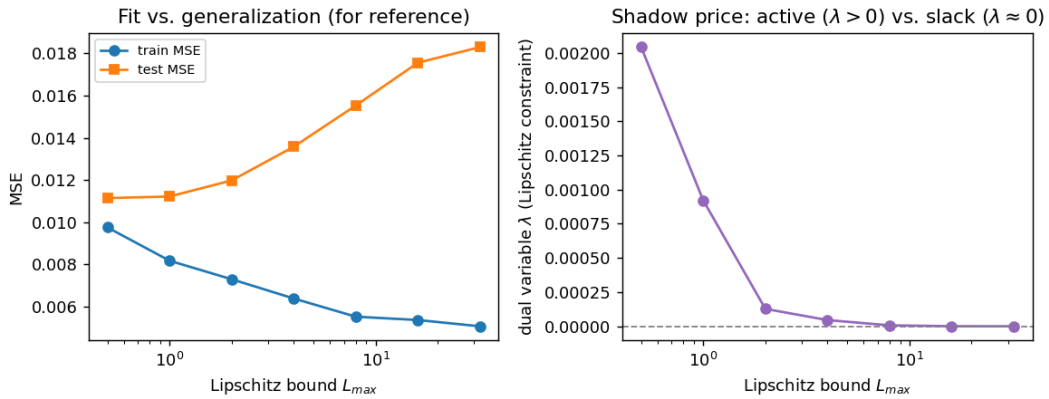


Fig. 3. KKT dual variable (shadow price) of the Lipschitz constraint vs L_{max} . $\lambda > 0$ confirms the constraint is binding; $\lambda \sim 0$ means it is slack.

4.4 NLP size scaling. Growing the hidden layer grows the NLP: from $H = 4$ (13 variables) to $H = 64$ (193 variables) the iteration count stays modest (127 to 328), but the solve time climbs from 0.194 s to 7.480 s -- about a 38x increase for a roughly 15x larger variable count. The growth is superlinear because each interior-point step factorizes a Hessian/KKT system whose cost scales worse than linearly in the problem dimension.

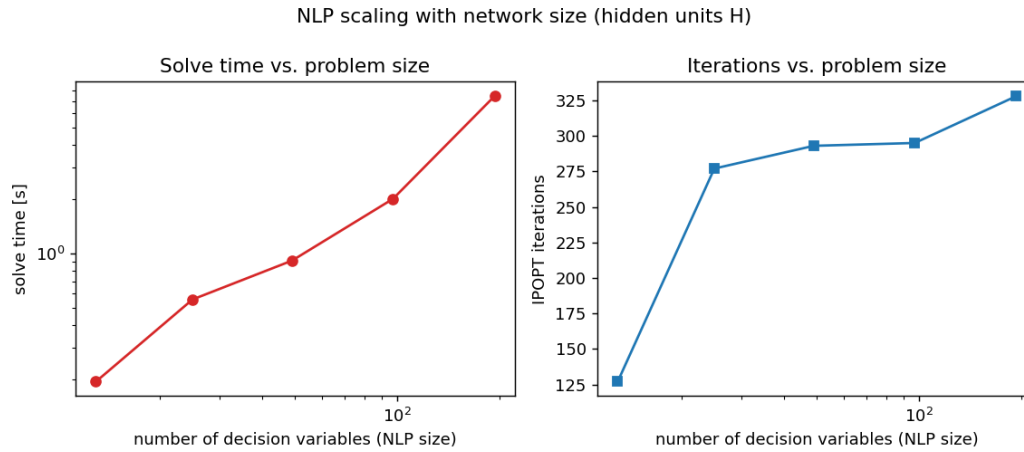


Fig. 4. Solve time and IPOPT iteration count vs NLP dimension (hidden units H). Superlinear growth in time reflects the cost of Hessian factorization.

4.5 Noise robustness. Repeating the constrained IPOPT solve and the unconstrained Adam run at increasing label noise isolates the regularization value of the hard constraints. At the highest noise level ($\sigma = 0.3$) the constrained model attains test MSE 0.10055 versus Adam's 0.10958, and crucially the unconstrained solution lets the network's Lipschitz value blow up to 11.506 while the constrained one is held at 4.000. The hard bound stops the model from fitting the noise.

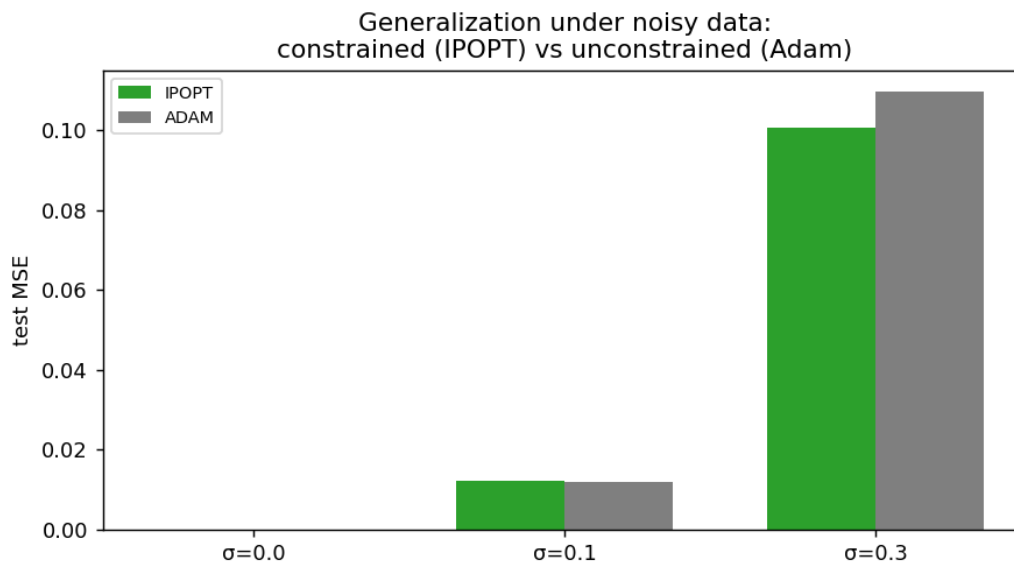


Fig. 5. Train/test MSE vs noise level for constrained IPOPT and unconstrained Adam. Hard constraints provide a regularization benefit at higher noise.

4.6 Penalty versus hard constraint. The standard ML way to honour a constraint is a soft penalty term. With no penalty the Lipschitz constraint is violated by 1.539; raising the penalty weight ρ shrinks the violation (to $2.77e-04$ at $\rho = 0.0001$) but never reaches zero and begins to distort the fit. IPOPT, by contrast, enforces the bound exactly -- violation $0.00e+00$ -- at test MSE 0.00286. Exact enforcement is the only way to guarantee feasibility.

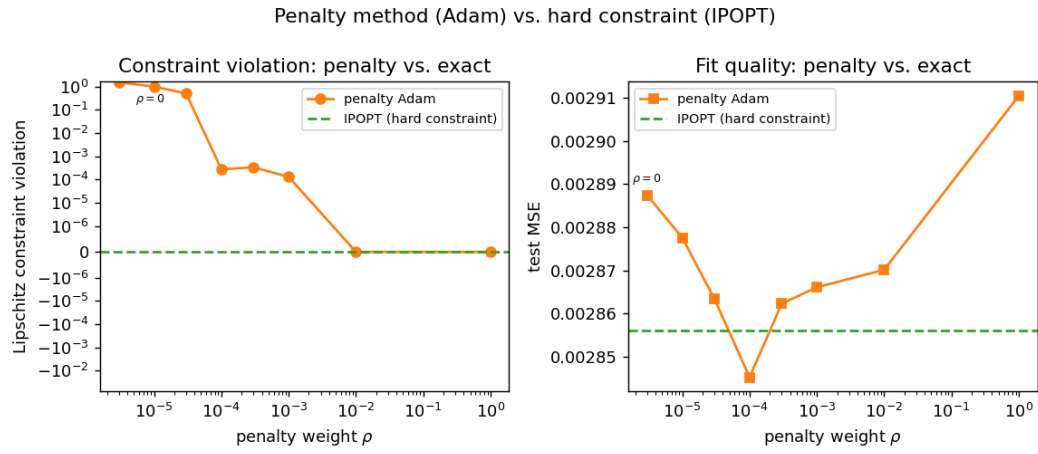


Fig. 6. Penalty Adam vs exact IPOPT enforcement. As ρ increases, penalty Adam reduces constraint violation but at the cost of fit quality. IPOPT enforces zero violation by construction.

5. Discussion and Conclusion

What worked. Posing training as a constrained NLP and solving it with IPOPT worked cleanly and gave information no first-order method provides: exact constraint satisfaction, shadow prices that quantify how much each bound costs, and a controllable fit/generalization dial through $L_{\max}$. The CasADi-supplied exact Hessian made IPOPT markedly more iteration-efficient than BFGS-based SQP on the same problem.

Nonconvexity. The bilinear Lipschitz constraint makes the problem nonconvex, but the 20-start study was reassuring: across all 20 random initializations the final training MSE ranged only from 0.00121 to 0.00141 (a spread of $2.04\text{e-}04$). The landscape has multiple stationary points -- visible as different iteration counts and basins -- yet they are of nearly equivalent quality here, so multistart is cheap insurance rather than a necessity at this scale.

Limitations and scaling. The main limitation is cost. Section 4.4 shows solve time growing superlinearly with the weight count because every interior-point step factorizes a dense KKT system. For the small networks studied here this is irrelevant -- seconds at most -- but it makes exact second-order constrained solves impractical for the million-parameter networks of mainstream deep learning, where Adam's cheap iterations win. The method's niche is small-to-medium models where guaranteed constraint satisfaction matters more than raw scale, such as certified-Lipschitz or safety-constrained networks.

Penalty versus hard. The penalty comparison was the clearest practical lesson: a soft penalty can be tuned to *reduce* a violation but never to eliminate it, and pushing the penalty weight up starts to corrupt the objective. When a bound must hold exactly -- a true Lipschitz certificate, a hard resource budget -- only explicit constraint handling delivers it, and IPOPT does so by construction.

Conclusion. Treating neural-network training as a constrained NLP turns implicit modelling choices into explicit, exactly-enforced constraints with interpretable dual information. On a controlled teacher-student task the constrained solver matched or beat the unconstrained baseline in generalization, enforced its bounds exactly where a penalty could not, and revealed -- through shadow prices and scaling curves -- precisely when each constraint mattered and

what it cost. The approach is a powerful analysis and design tool for the small-network regime, and a clarifying lens on what regularization really does.

References

- [1] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi: a software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [2] A. Wächter and L. T. Biegler, "On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming," *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.